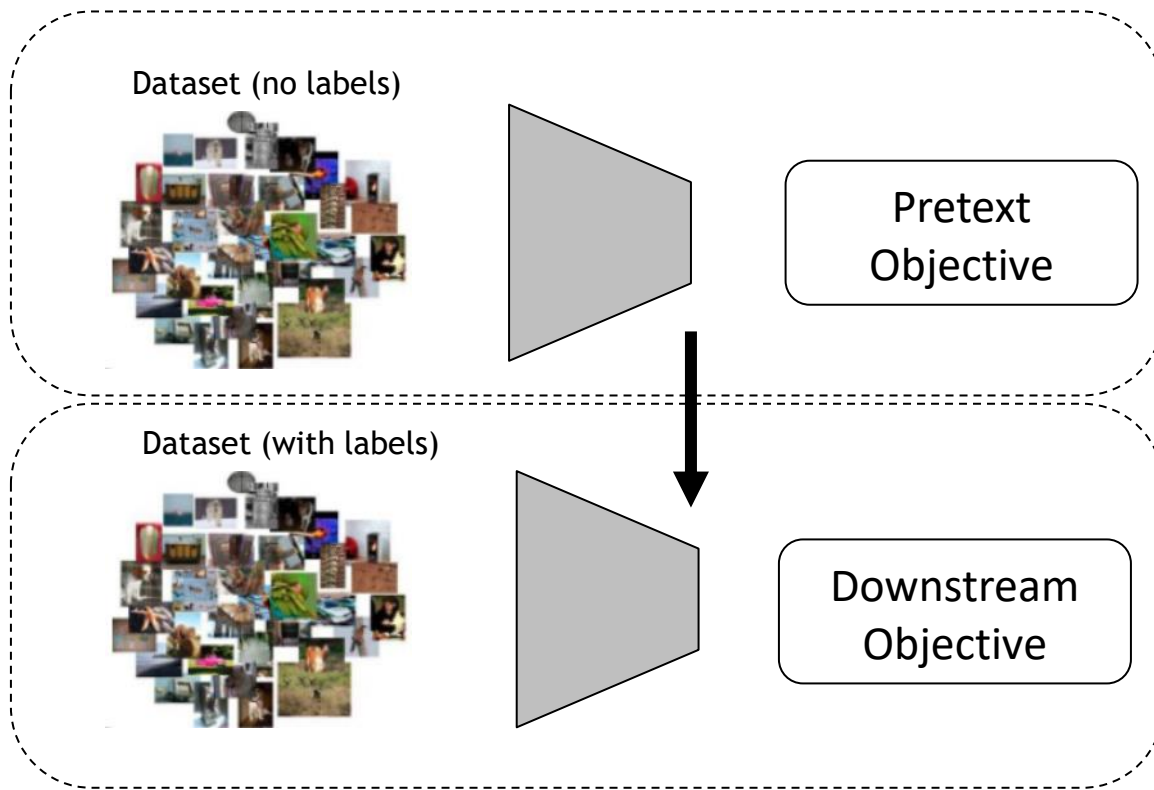


Today's Outline

- Self-Supervised Pre-training

Self-Supervised Learning



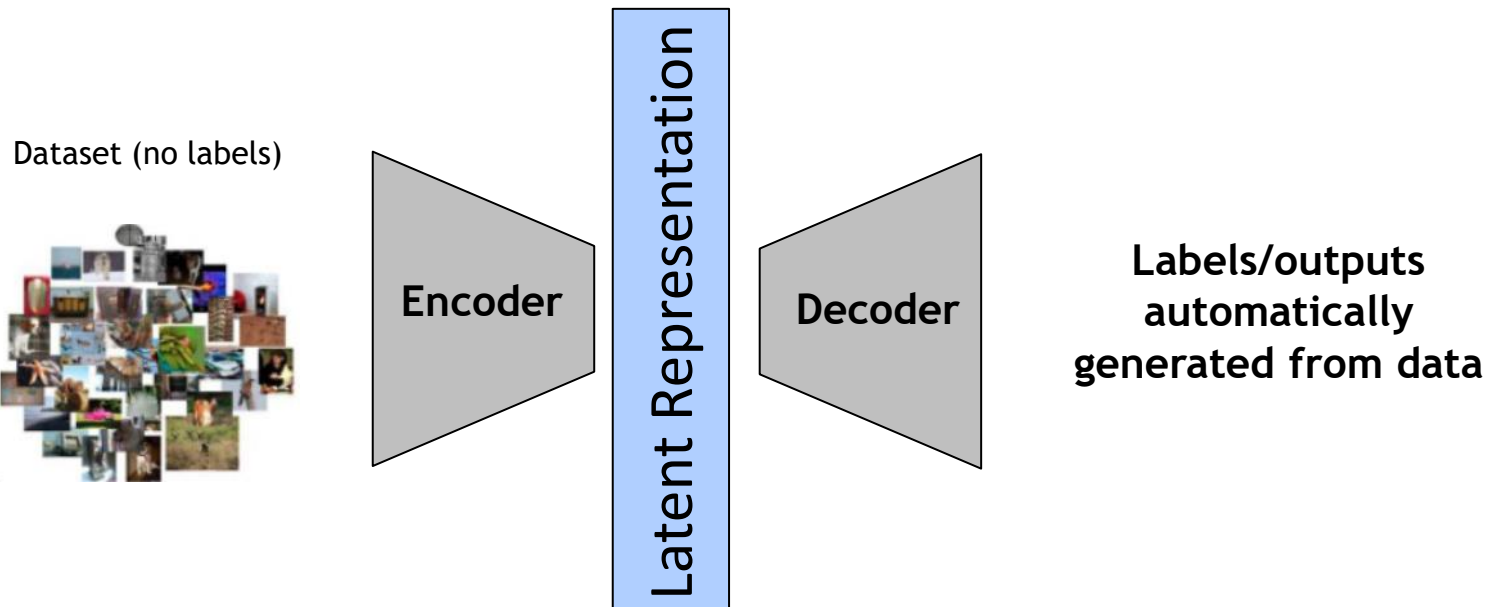
Pretext Task

- Define a task based on the data itself
- No manual annotation
- Could be considered an unsupervised task;
- but we learn with supervised learning objectives, e.g., classification or regression.

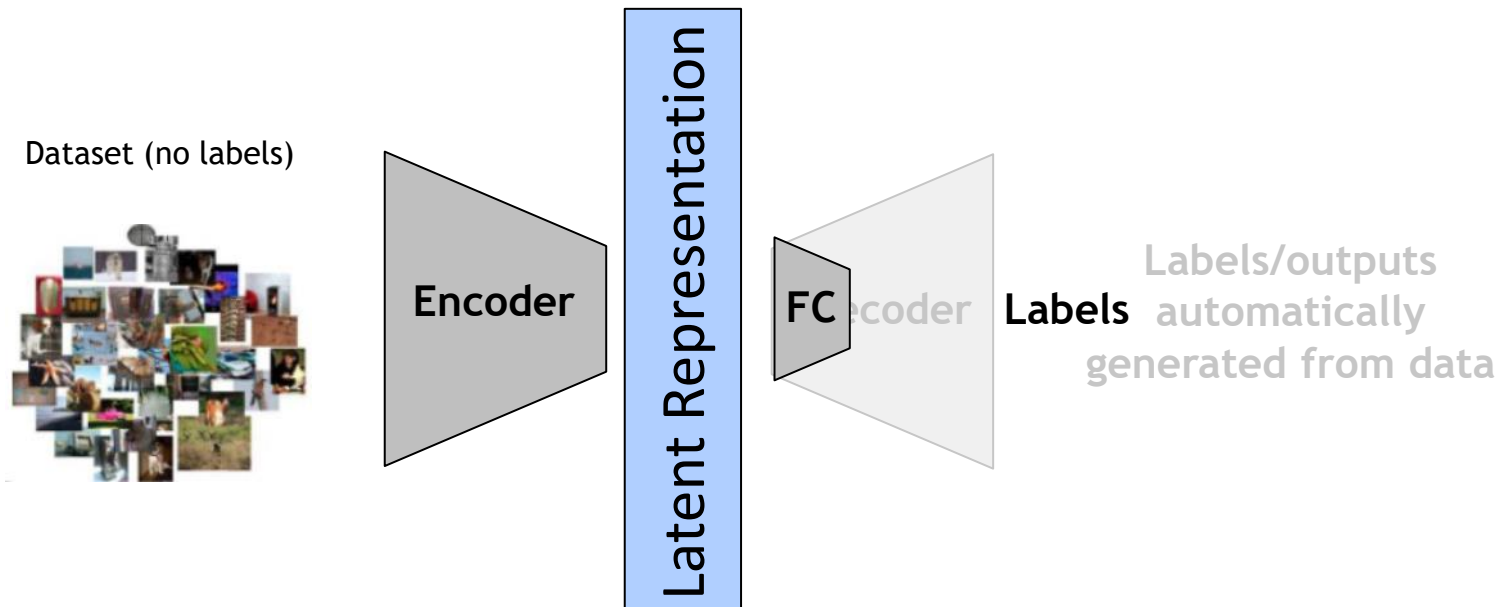
Downstream Task

- The application you care about
- You do not have large datasets
- The dataset is labeled

Self-Supervised Learning - Pretext Task



Self-Supervised Learning - Downstream Task

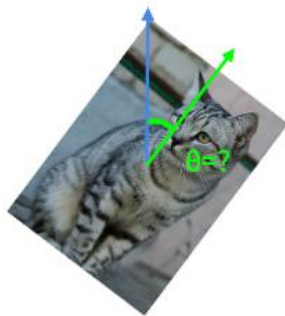


Self-supervised pretext tasks

Example: learn to predict image transformations / complete corrupted images



image
completion



rotation
prediction



“jigsaw
puzzle”



colorization



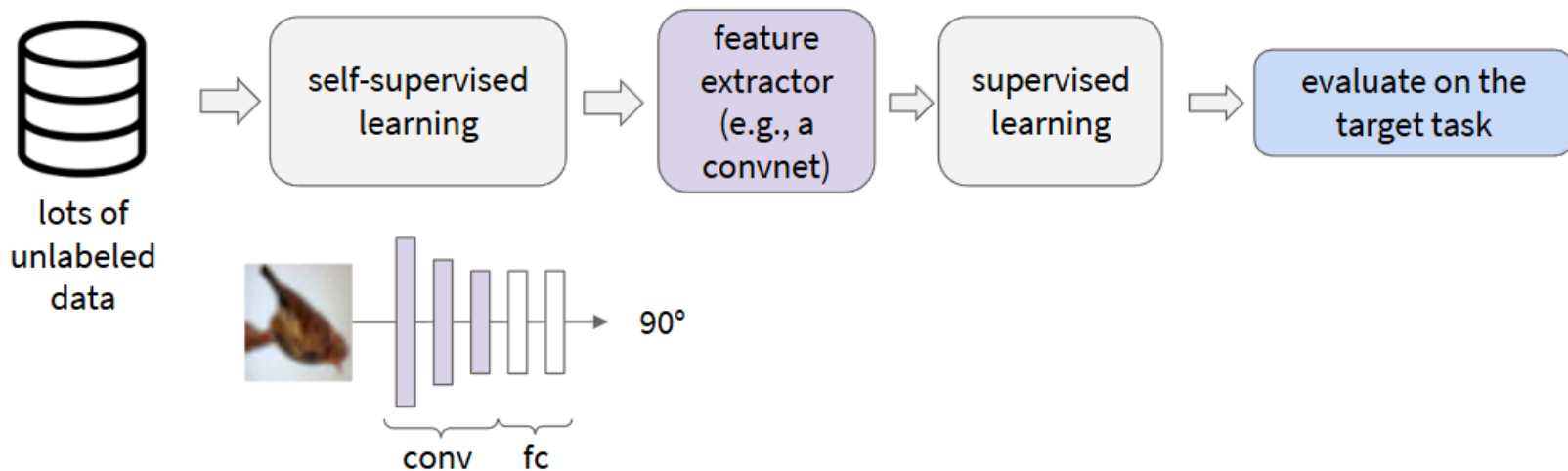
novel view
synthesis

1. Solving the pretext tasks allow the model to learn good features.
2. We can automatically generate labels for the pretext tasks.

How to evaluate a self-supervised learning method?

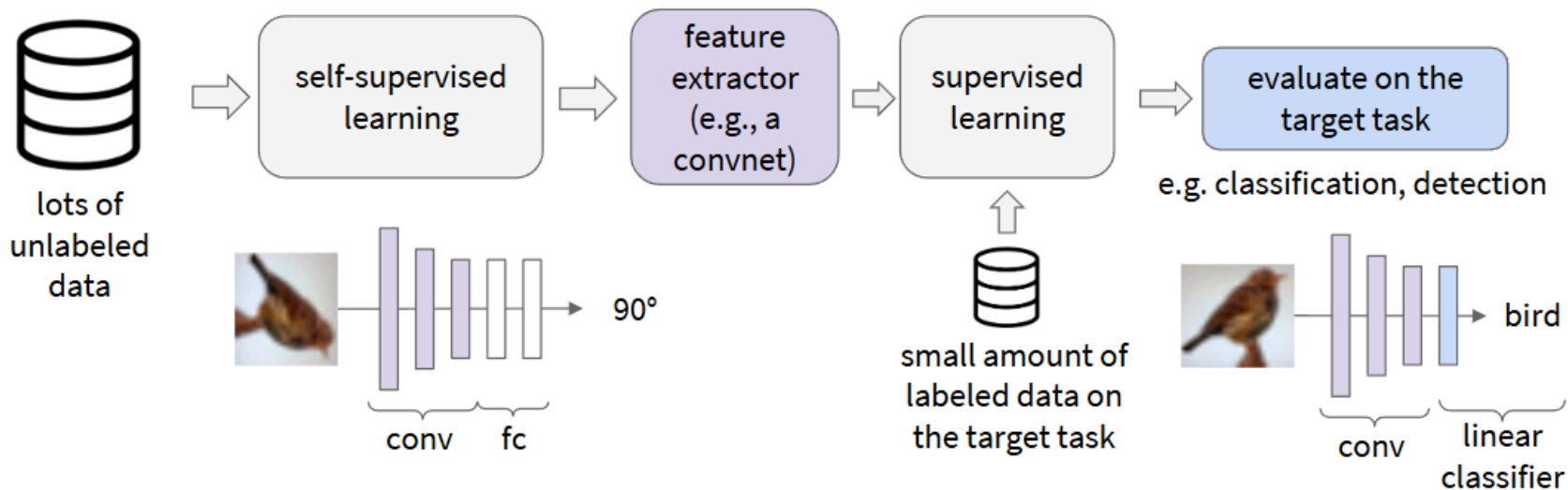
- Pretext Task Performance
 - Measure how well the model performs on the task it was trained on without labels.
- Representation Quality
 - Evaluate the quality of the learned representations
 - Linear Evaluation Protocol: Train a linear classifier on the learned representations;
 - Clustering: Measure clustering performance;
 - t-SNE: Visualize the representations to assess their separability.
- Robustness and Generalization
 - Test how well the model generalizes to different datasets and is robust to variations.
- Computational Efficiency
 - Assess the efficiency of the method in terms of training time and resource requirements.
- Transfer Learning and Downstream Task Performance
 - Assess the utility of the learned representations by transferring them to a downstream supervised task.

How to evaluate a self-supervised learning method?



1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

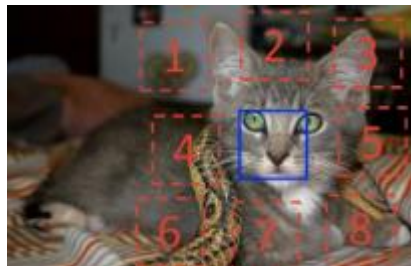
How to evaluate a self-supervised learning method?



1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

2. Attach a shallow network on the feature extractor; train the shallow network on the target task with small amount of labeled data

Broader picture



computer vision



robot / reinforcement
learning

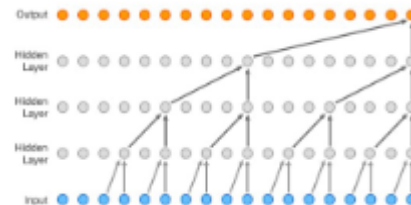
GPT-4 Technical Report

OpenAI*

Abstract

We report the development of GPT-4, a large-scale, multimodal model which can accept image and text inputs and produce text outputs. While less capable than humans in many real-world scenarios, GPT-4 exhibits human-level performance on various professional and academic benchmarks, including passing a simulated bar exam with a score around the top 10% of test takers. GPT-4 is a Transformer-based model pre-trained to predict the next token in a document. The post-training alignment process results in improved performance on measures of factuality and adherence to desired behavior. A core component of this project was developing infrastructure and optimization methods that behave predictably across a wide range of scales. This allowed us to accurately predict some aspects of GPT-4's performance based on models trained with no more than 1/1,000th the compute of GPT-4.

language modeling



speech synthesis

Today's Agenda

- Pretext tasks from 2D image transformations
 - Rotation, inpainting, rearrangement, coloring
- Pretext tasks from 3D image transformations
 - Novel view synthesis
- Contrastive representation learning
 - Intuition and formulation; Several representative method
- Pretext tasks from annotated video

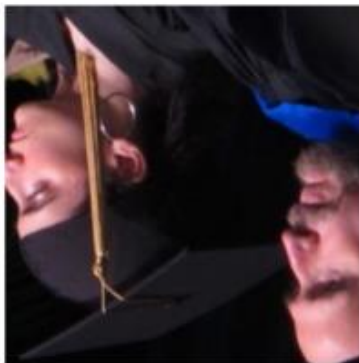
Pretext task: predict rotations in 2D images



90° rotation



270° rotation



180° rotation



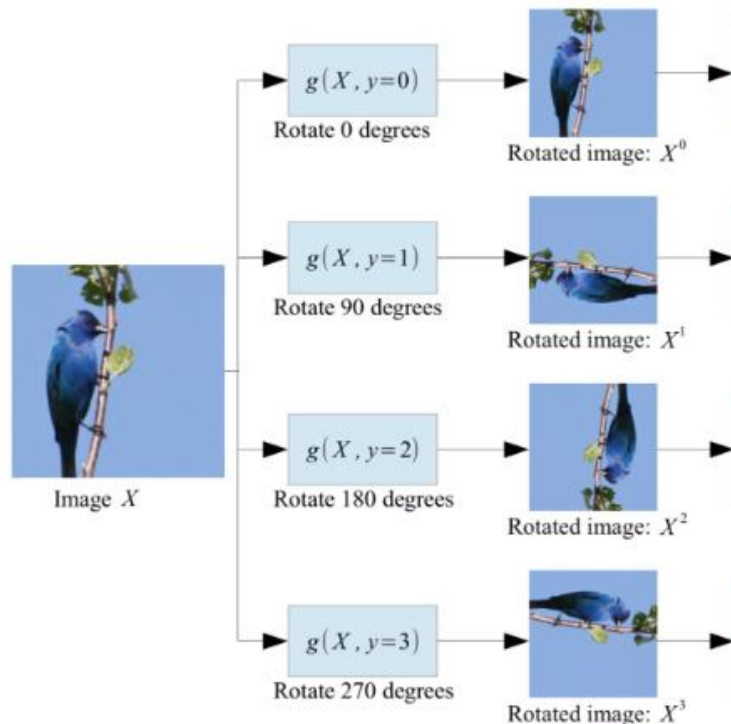
0° rotation



270° rotation

Hypothesis: a model could recognize the correct rotation of an object only if it has the “visual commonsense” of what the object should look like unperturbed.

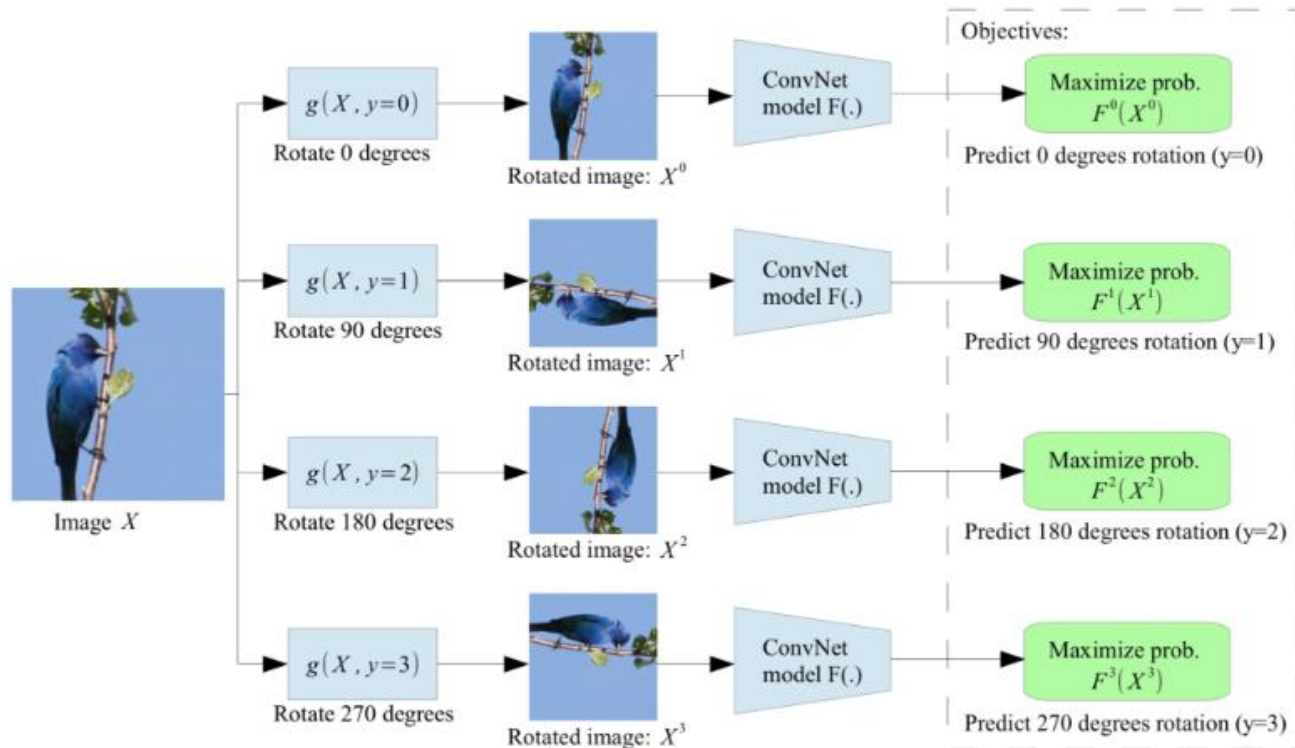
Pretext task: predict rotations in 2D images



Self-supervised learning by rotating the entire input images.

The model learns to predict which rotation is applied (4-way classification)

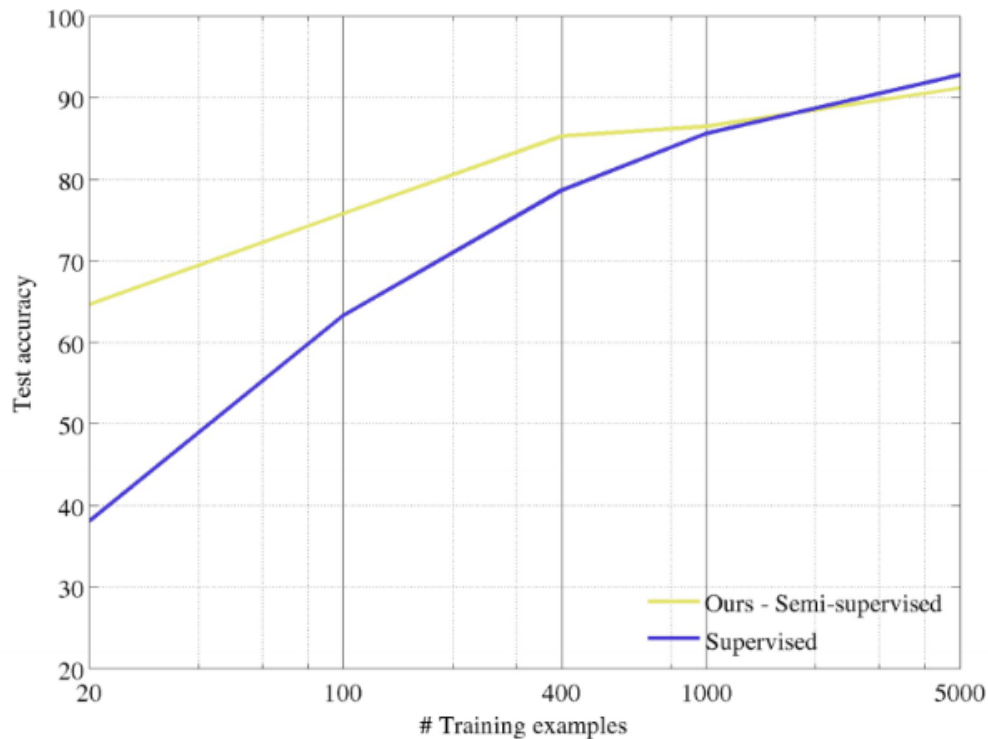
Pretext task: predict rotations in 2D images



Self-supervised learning by rotating the entire input images.

The model learns to predict which rotation is applied (4-way classification)

Evaluation on semi-supervised learning



Self-supervised learning on CIFAR10 (entire training set).

Freeze conv1 + conv2
Learn conv3 + linear layers with subset of labeled CIFAR10 data (classification).

Transfer learned features to supervised learning

	Classification (%mAP)		Detection (%mAP)	Segmentation (%mIoU)
Trained layers	fc6-8	all	all	all
ImageNet labels	78.9	79.9	56.8	48.0
Random		53.3	43.4	19.8
Random rescaled Krähenbühl et al. (2015)	39.2	56.6	45.6	32.6
Egomotion (Agrawal et al., 2015)	31.0	54.2	43.9	
Context Encoders (Pathak et al., 2016b)	34.6	56.5	44.5	29.7
Tracking (Wang & Gupta, 2015)	55.6	63.1	47.4	
Context (Doersch et al., 2015)	55.1	65.3	51.1	
Colorization (Zhang et al., 2016a)	61.5	65.6	46.9	35.6
BIGAN (Donahue et al., 2016)	52.3	60.1	46.9	34.9
Jigsaw Puzzles (Noroozi & Favaro, 2016)	-	67.6	53.2	37.6
NAT (Bojanowski & Joulin, 2017)	56.7	65.3	49.4	
Split-Brain (Zhang et al., 2016b)	63.0	67.1	46.7	36.0
ColorProxy (Larsson et al., 2017)		65.9		38.4
Counting (Noroozi et al., 2017)	-	67.7	51.4	36.6
(Ours) RotNet	70.87	72.97	54.4	39.1

Pretrained with full ImageNet supervision

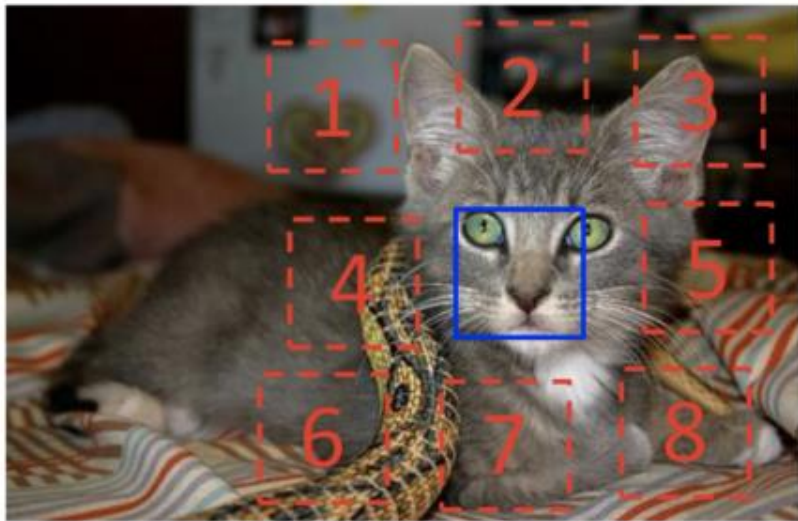
No pretraining

Self-supervised learning on ImageNet (entire training set) with AlexNet.

Finetune on labeled data from Pascal VOC 2007.

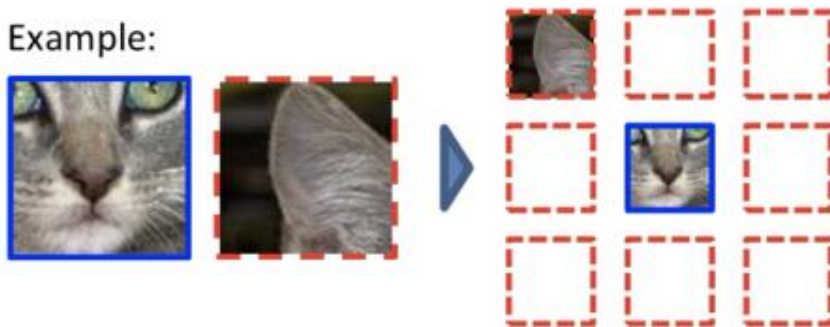
Self-supervised learning with rotation prediction

Pretext task: predict relative patch locations



$$X = (\text{cat face patch}, \text{cat ear patch}); Y = 3$$

Example:



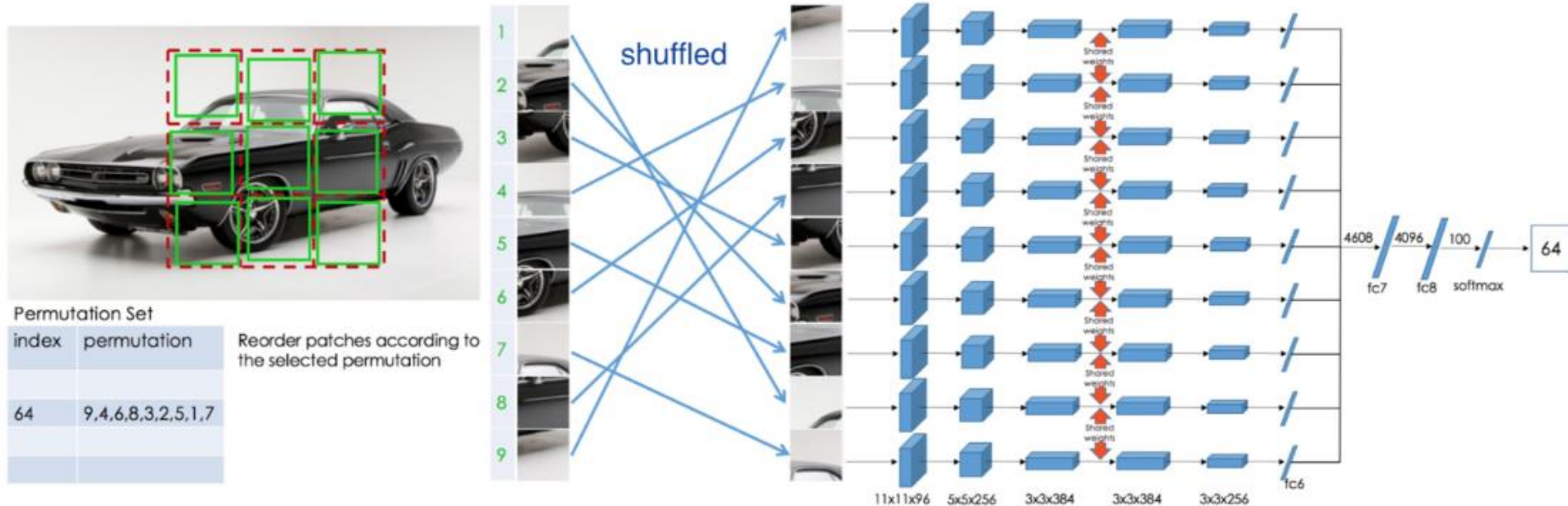
Question 1:



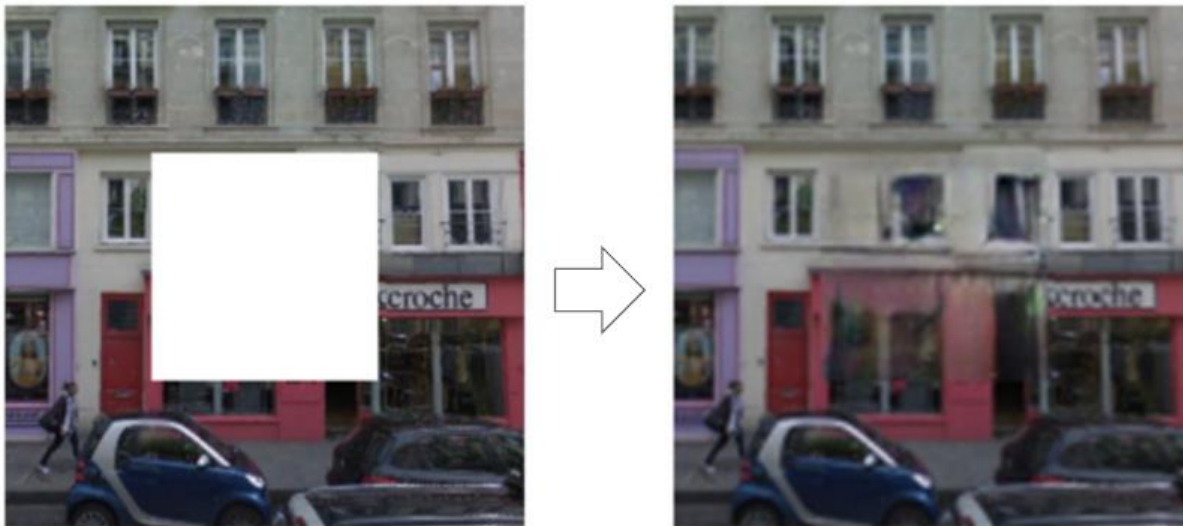
Question 2:



Pretext task: solving “jigsaw puzzles”

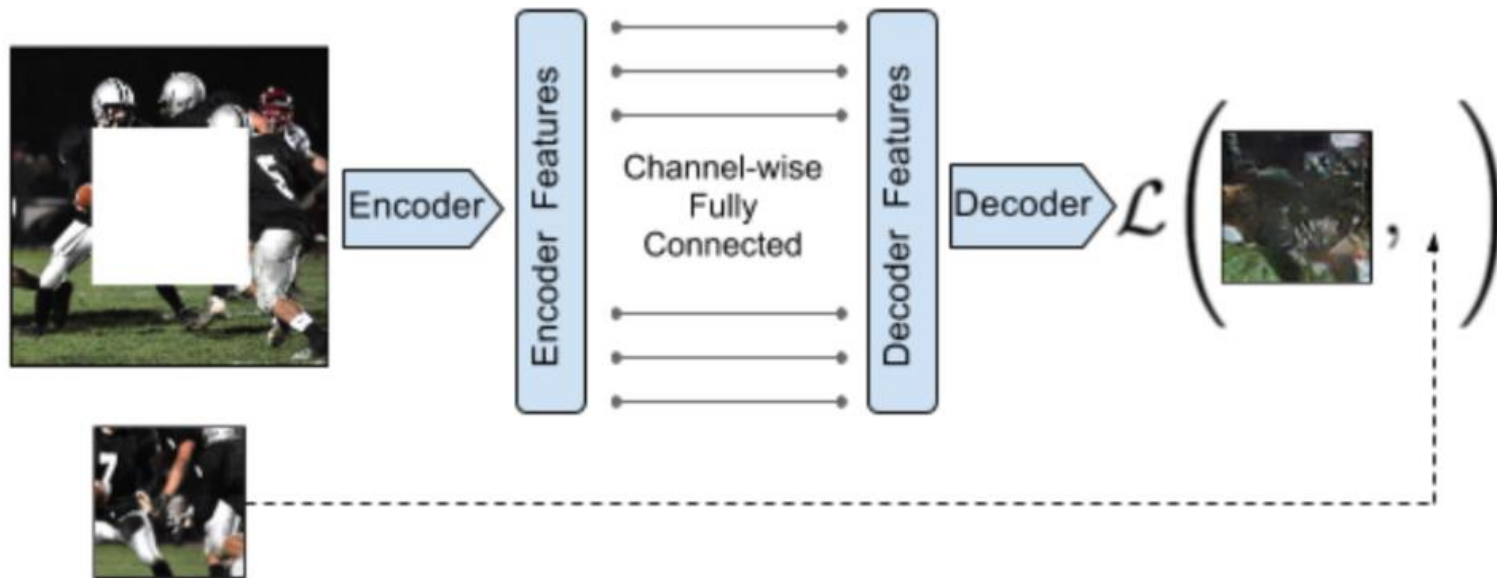


Pretext task: predict missing pixels (inpainting)



Context Encoders: Feature Learning by Inpainting

Pretext task: predict missing pixels (inpainting)



Learning to reconstruct the missing pixels

Inpainting evaluation



Input (context)



reconstruction

Learning to inpaint by reconstruction

Loss = reconstruction + adversarial learning

$$L(x) = L_{recon}(x) + L_{adv}(x)$$

$$L_{recon}(x) = ||M * (x - F_{\theta}((1 - M) * x))||_2^2$$

$$L_{adv} = \max_D \mathbb{E}[\log(D(x))] + \log(1 - D(F((1 - M) * x)))]$$

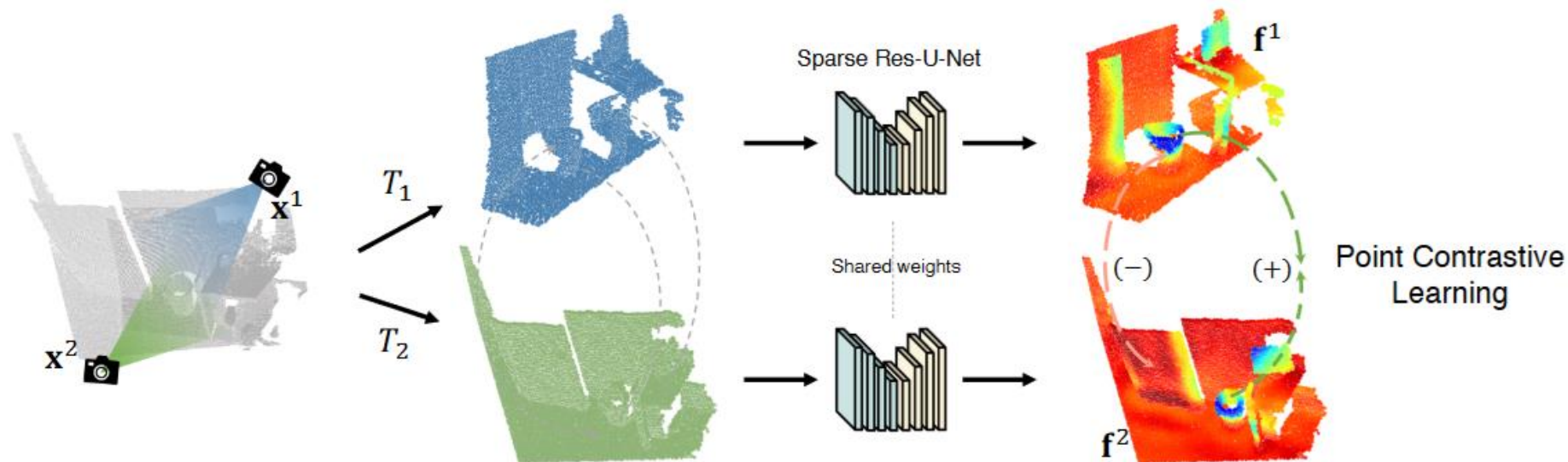
Adversarial loss between “real” images and inpainted images

Transfer learned features to supervised learning

Pretraining Method	Supervision	Pretraining time	Classification	Detection	Segmentation
ImageNet [26]	1000 class labels	3 days	78.2%	56.8%	48.0%
Random Gaussian	initialization	< 1 minute	53.3%	43.4%	19.8%
Autoencoder	-	14 hours	53.8%	41.9%	25.2%
Agrawal <i>et al.</i> [1]	egomotion	10 hours	52.9%	41.8%	-
Wang <i>et al.</i> [39]	motion	1 week	58.7%	47.4%	-
Doersch <i>et al.</i> [7]	relative context	4 weeks	55.3%	46.6%	-
Ours	context	14 hours	56.5%	44.5%	30.0%

Self-supervised learning on ImageNet training set, transfer to classification (Pascal VOC 2007), detection (Pascal VOC 2007), and semantic segmentation (Pascal VOC 2012)

Unsupervised Pre-training for 3D Data



Methodology for pretraining task.

Unsupervised Pre-training for 3D Data

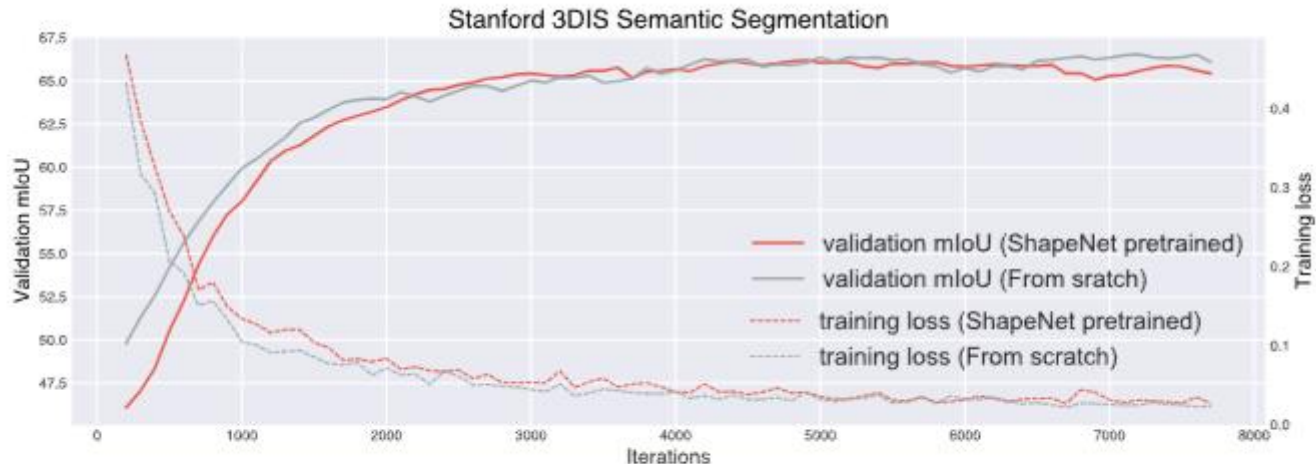
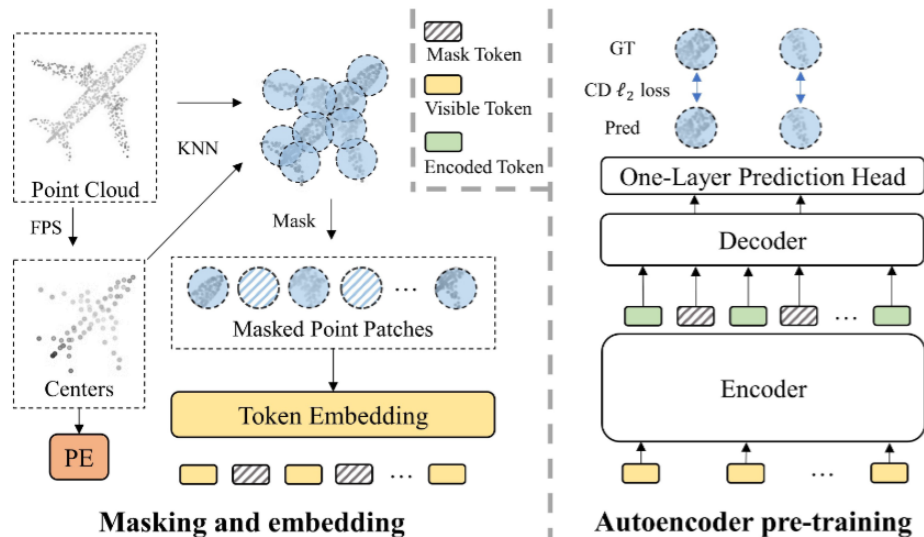
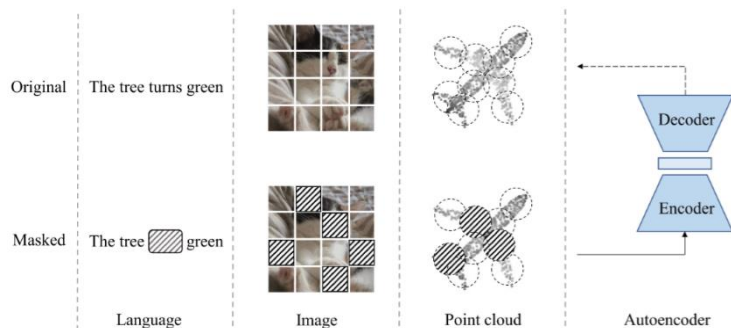
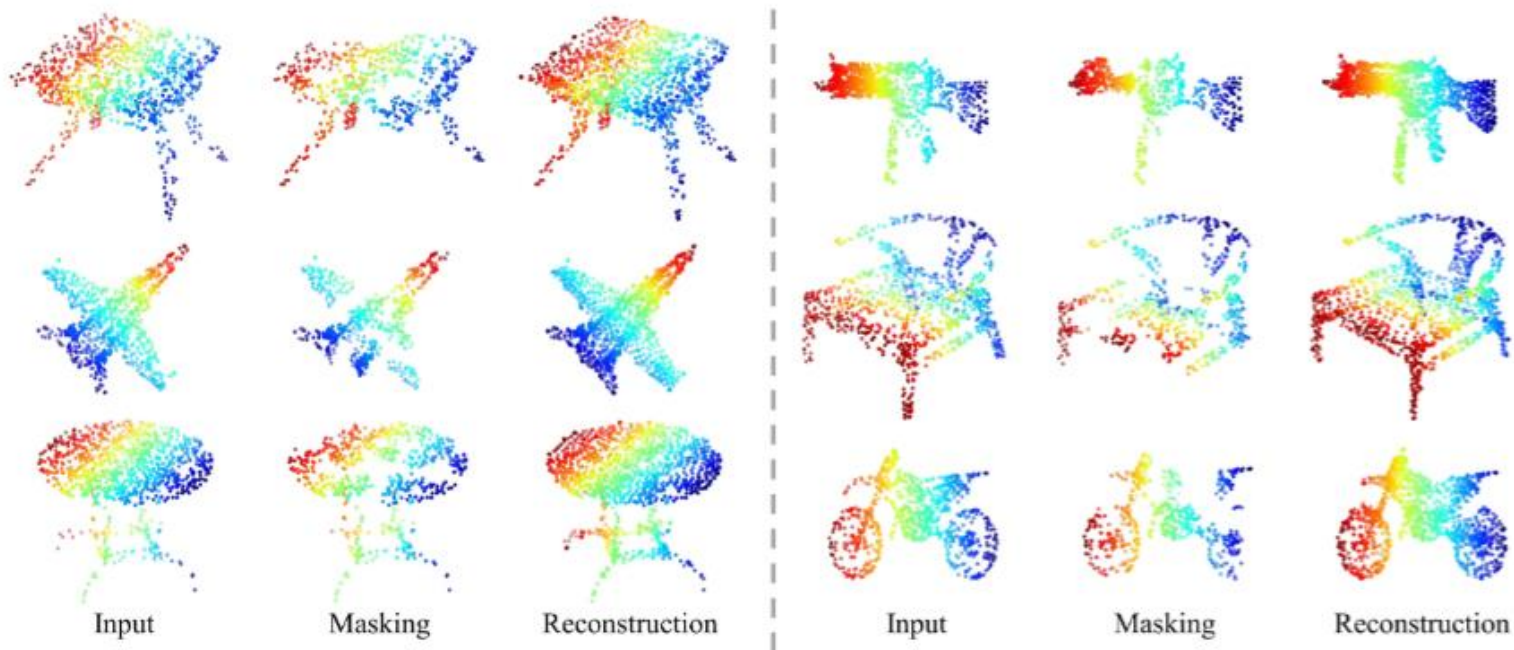


Fig. 1: Training from scratch *vs.* fine-tuning with ShapeNet pre-trained weights.

Unsupervised Pre-training for 3D Data: Masked VAE



Unsupervised Pre-training for 3D Data: Masked VAE



Reconstruction results after Masked MAE pretraining.

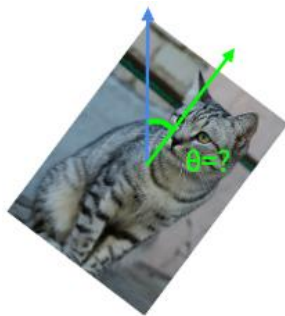
Summary: pretext tasks from image/3D transformations

- Pretext tasks focus on “visual common sense”, e.g., predict rotations, inpainting, rearrangement, and colorization.
- The models are forced learn good features about natural images, e.g., semantic representation of an object category, in order to solve the pretext tasks.
- We often do not care about the performance of these pretext tasks, but rather how useful the learned features are for downstream tasks(classification, detection, segmentation).
- Problems: 1) coming up with individual pretext tasks is tedious, and 2) the learned representations may not be general.

Summary: pretext tasks from image transformations



image
completion



rotation
prediction



"jigsaw
puzzle"



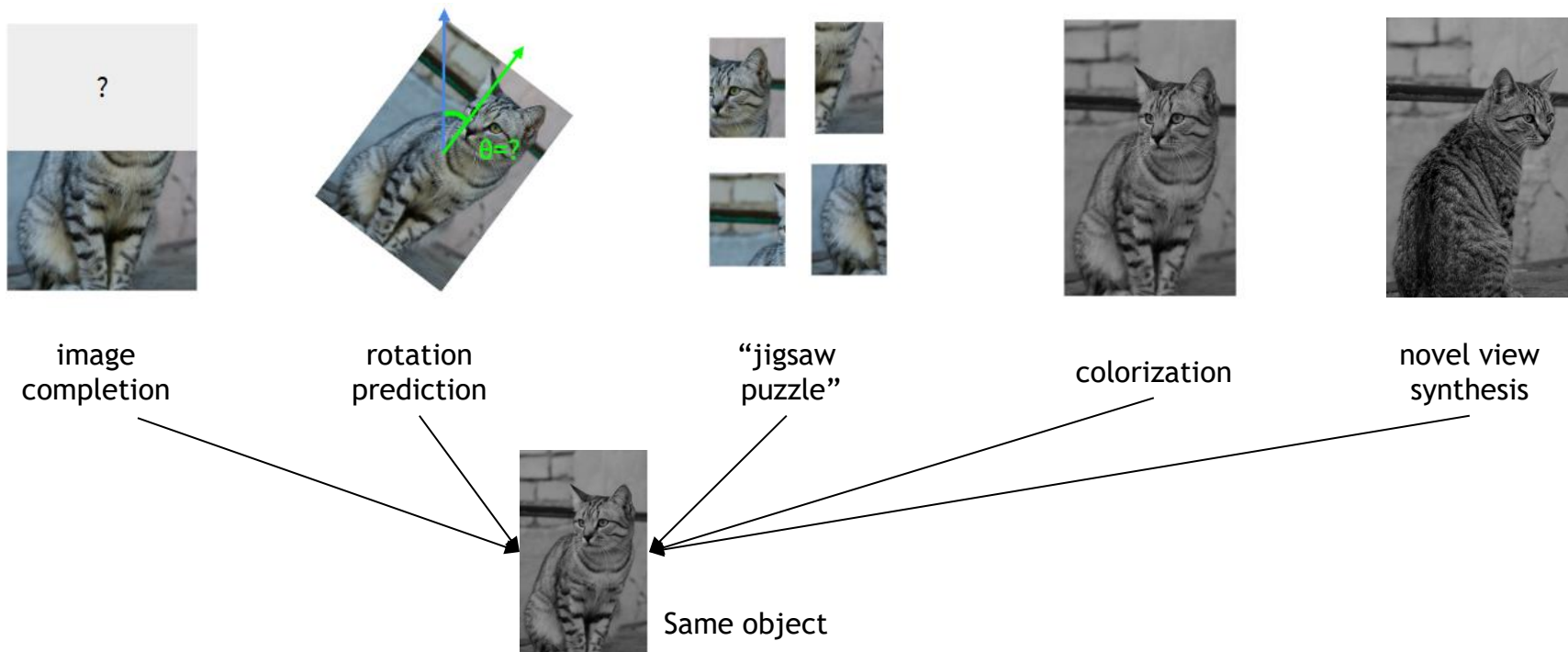
colorization



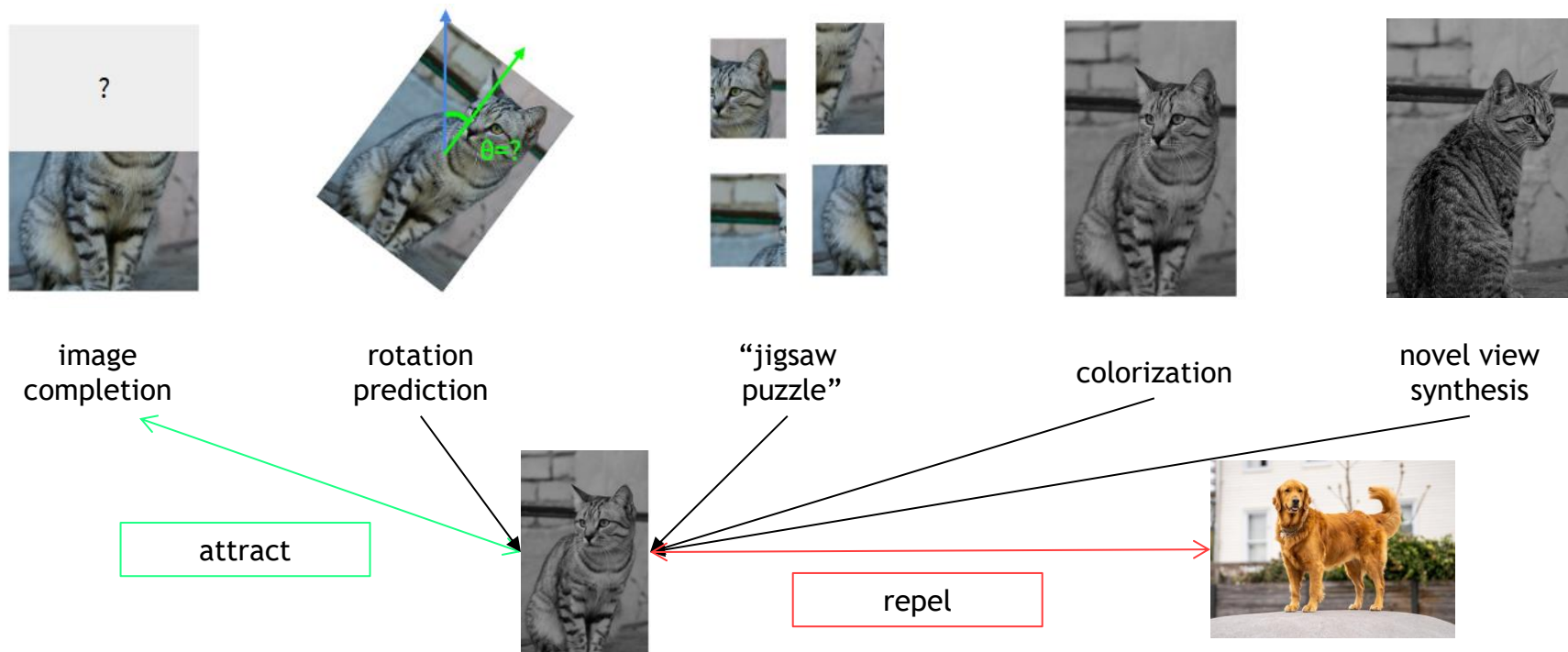
novel view
synthesis

1. Learned representations may be tied to a specific pretext task!
2. Can we come up with a more general pretext task?

Summary: pretext tasks from image transformations



A more general pretext task? Contrastive Learning



A formulation of contrastive learning

- What we want:

$$\text{score}(f(x), f(x^+)) \gg \text{score}(f(x), f(x^-))$$

- x : reference sample; x^+ positive sample; x^- negative sample
- Given a chosen score function, we aim to learn an encoder function f that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-) .

A formulation of contrastive learning

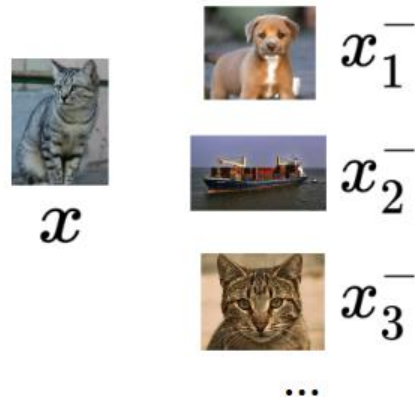
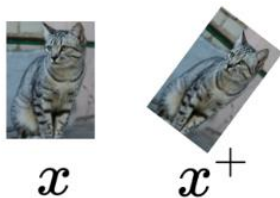
- Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

A formulation of contrastive learning

- Loss function given 1 positive sample and N - 1 negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$



A formulation of contrastive learning

- Loss function given 1 positive sample and N - 1 negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}}{\underbrace{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \underbrace{\exp(s(f(x), f(x_j^-)))}}_{\text{score for the N-1 negative pairs}}} \right]$$

score for the
positive pair

score for the
N-1 negative pairs

- This seems familiar ... Cross entropy loss for a N-way softmax classifier! i.e., learn to find the positive sample from the N samples

A formulation of contrastive learning

- Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}^{\text{score for the positive pair}}}{\underbrace{\exp(s(f(x), f(x^+)))}_{\text{score for the positive pair}} + \sum_{j=1}^{N-1} \underbrace{\exp(s(f(x), f(x_j^-)))}_{\text{score for the N-1 negative pairs}}} \right]$$

score for the
positive pair

score for the
N-1 negative pairs

- Commonly known as the InfoNCE loss
- A lower bound on the mutual information between $f(x)$ and $f(x^+)$

$$MI[f(x), f(x^+)] - \log(N) \geq -L$$

- The larger the negative sample size (N), the tighter the bound

SimCLR: A Simple Framework for Contrastive Learning

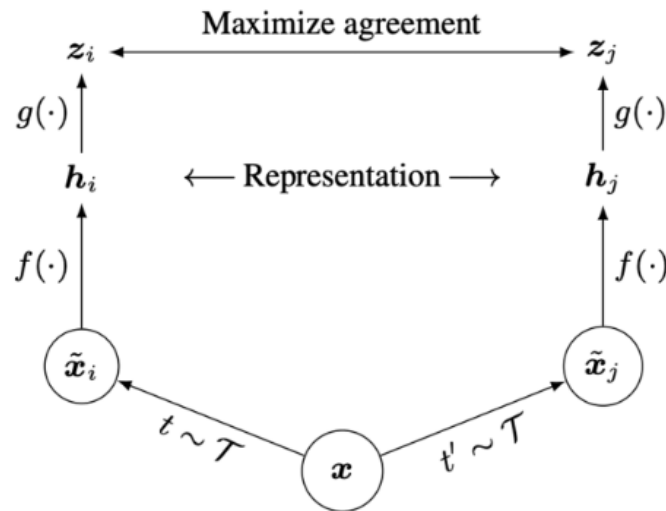
Cosine similarity as the score function:

$$s(u, v) = \frac{u^T v}{||u|| ||v||}$$

Use a projection network $g(\cdot)$ to project features to a space where contrastive learning is applied

Generate positive samples through data augmentation:

- random cropping, random color distortion, and random blur.



SimCLR: A Simple Framework for Contrastive Learning



(a) Original



(b) Crop and resize



(c) Crop, resize (and flip)



(d) Color distort. (drop)



(e) Color distort. (jitter)



(f) Rotate $\{90^\circ, 180^\circ, 270^\circ\}$



(g) Cutout



(h) Gaussian noise



(i) Gaussian blur



(j) Sobel filtering

SimCLR

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N , constant τ , structure of $f, g, \mathcal{T}$.

for sampled minibatch $\{\mathbf{x}_k\}_{k=1}^N$ **do**

for all $k \in \{1, \dots, N\}$ **do**

 draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$

 # the first augmentation

$\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$

$\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$

 # representation

$\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$

 # projection

 # the second augmentation

$\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$

$\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$

 # representation

$\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$

 # projection

end for

for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**

$s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ # pairwise similarity

end for

define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$

$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$

 update networks f and g to minimize $\mathcal{L}$

end for

return encoder network $f(\cdot)$, and throw away $g(\cdot)$

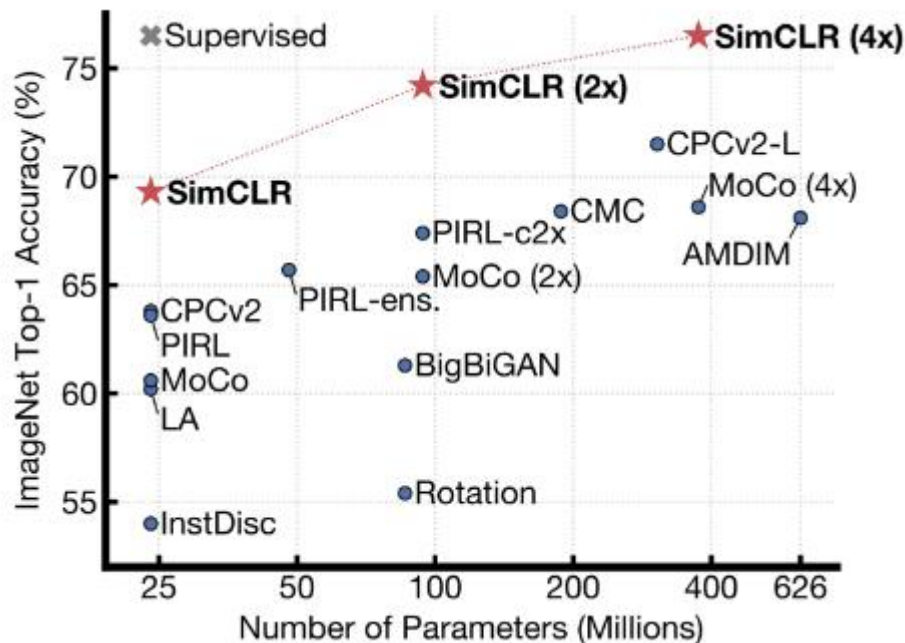
Generate a positive pair
by sampling data
augmentation functions

Iterate through and use
each of the $2N$ sample
as reference, compute
average loss

*We use a slightly different
formulation in the
assignment. You should follow
the assignment instructions.

InfoNCE loss:
Use all non-positive
samples in the batch
as $\mathbf{x}^-$

Training linear classifier on SimCLR features



Train feature encoder on ImageNet(entire training set) using SimCLR.

Freeze feature encoder, train a linear classifier on top with labeled data.

Semi-supervised learning on SimCLR features

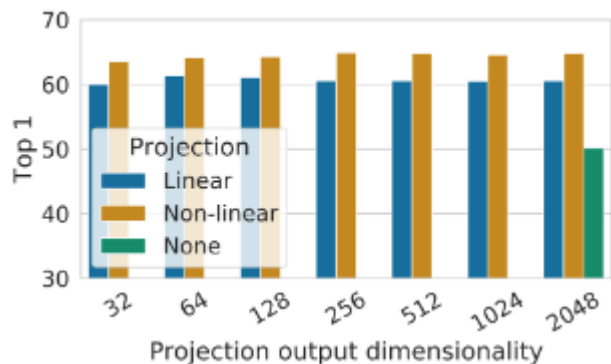
Method	Architecture	Label fraction	
		1%	10%
		Top 5	
Supervised baseline	ResNet-50	48.4	80.4
<i>Methods using other label-propagation:</i>			
Pseudo-label	ResNet-50	51.6	82.4
VAT+Entropy Min.	ResNet-50	47.0	83.4
UDA (w. RandAug)	ResNet-50	-	88.5
FixMatch (w. RandAug)	ResNet-50	-	89.1
S4L (Rot+VAT+En. M.)	ResNet-50 (4×)	-	91.2
<i>Methods using representation learning only:</i>			
InstDisc	ResNet-50	39.2	77.4
BigBiGAN	RevNet-50 (4×)	55.2	78.8
PIRL	ResNet-50	57.2	83.8
CPC v2	ResNet-161(*)	77.9	91.2
SimCLR (ours)	ResNet-50	75.5	87.8
SimCLR (ours)	ResNet-50 (2×)	83.0	91.2
SimCLR (ours)	ResNet-50 (4×)	85.8	92.6

Table 7. ImageNet accuracy of models trained with few labels.

Train feature encoder on ImageNet(entire training set) using SimCLR.

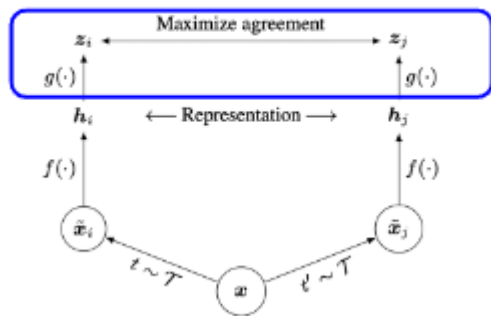
Finetune the encoder with 1% / 10%of labeled data on ImageNet.

Semi-supervised learning on SimCLR features



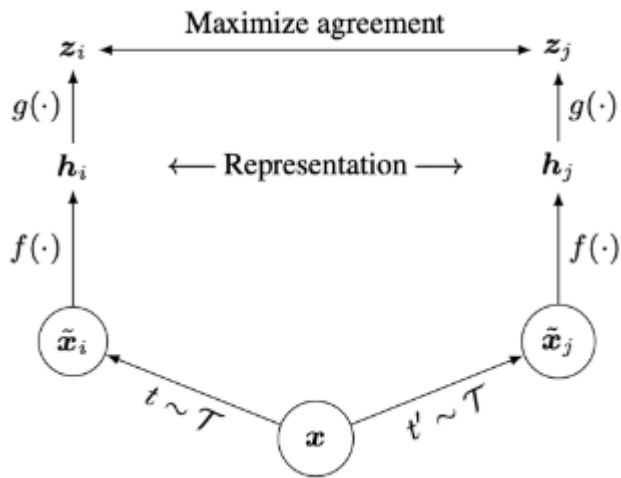
Linear / non-linear projection heads improve representation learning.

A possible explanation:



- contrastive learning objective may discard useful information for downstream tasks
- representation space z is trained to be invariant to data transformation
- by leveraging the projection head $g(\cdot)$, more information can be preserved in the h representation space

Summary: Contrastive Representation Learning

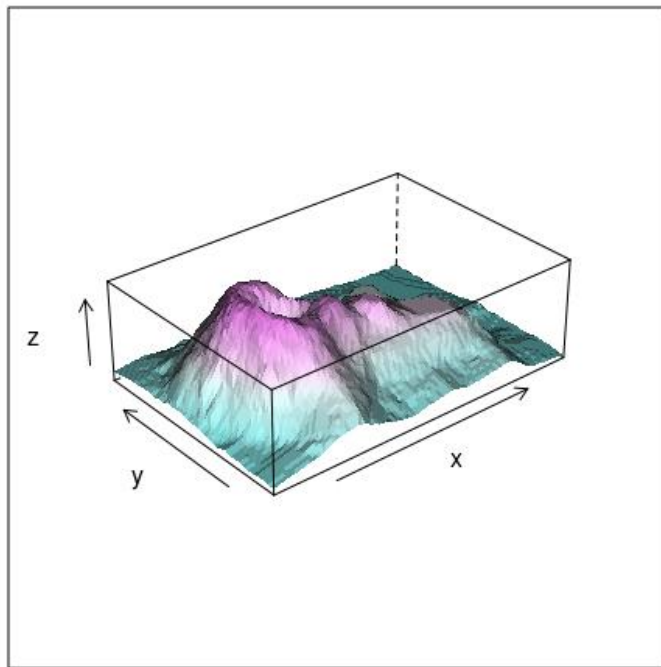


SimCLR: a simple framework for contrastive representation learning

A possible explanation:

- Key ideas: non-linear projection head to allow flexible representation learning
- Simple to implement, effective in learning visual representation
- Requires large training batch size to be effective; large memory footprint

Back to the 3D data processing



$(x, y, \mathbf{z})$

Self-Supervised Pretraining in 3D Tasks

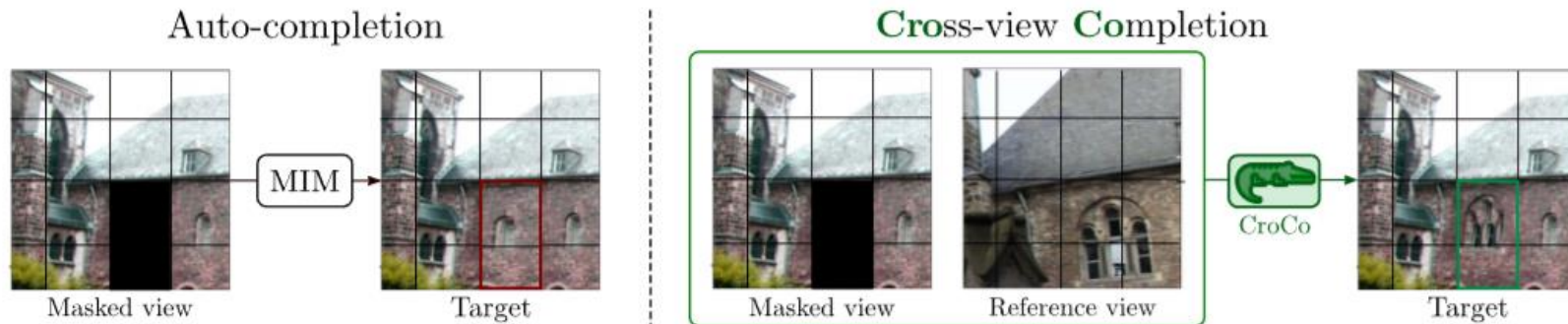
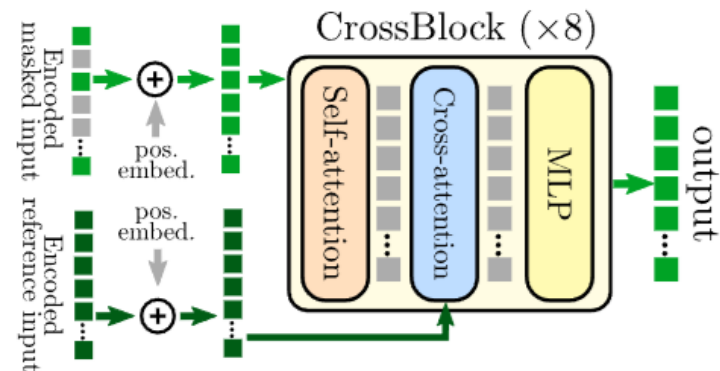
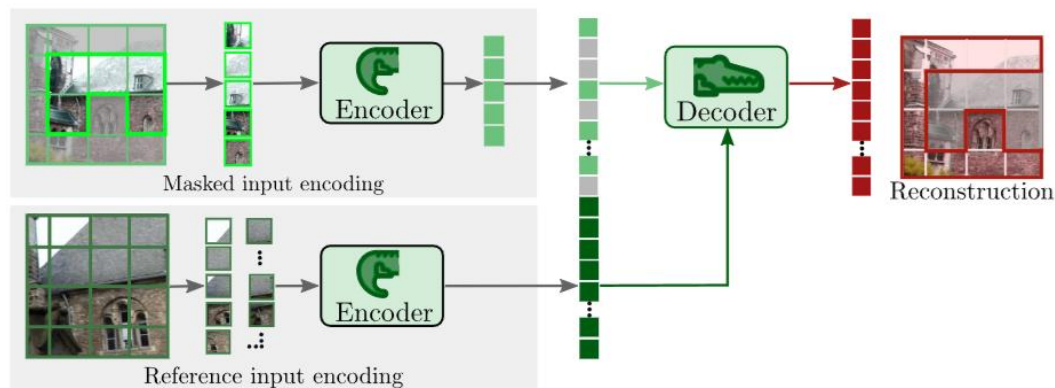


Figure 1: **Auto-completion vs. Cross-view completion tasks.** *Left:* given a masked image, a model trained for auto-completion can only leverage the visible context to fill-in the blanks and thus relies mainly on high-level semantic information. *Right:* given an additional view of the same scene, cross-view completion makes precise reconstruction possible, assuming that the model is able to understand both the scene geometry and the spatial relationship between the two images.

Self-Supervised Pretraining in 3D Tasks



Hyperparameter Selection

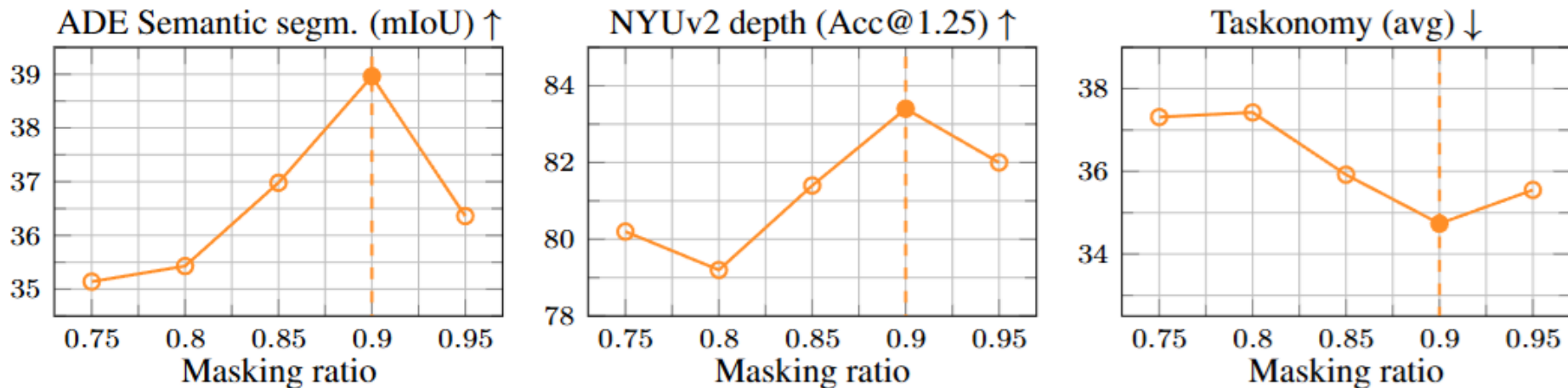


Figure 5: **Impact of the masking ratio** using the CrossBlock decoder and a loss on RGB values.

Downstream task validation

Table 3: **Comparison to the state-of-the-art pre-training methods** on semantic tasks (image classification on ImageNet-1K with linear probing and semantic segmentation on ADE) and 3D vision tasks (NYUv2, Taskonomy), with a ViT-Base/16 backbone. We indicate the pre-training data in parenthesis. Best and second best results are **bold** and underlined.

pre-training method (data)	IN1K $\uparrow$	ADE $\uparrow$	NYUv2 $\uparrow$	Taskonomy $\downarrow$									
	lin.	segm.	depth	curv.	depth	edges	kpts2d	kpts3d	normal	occl.	reshad.	avg.	rank.
DINO [14] (IN1K)	78.2	44.7	66.8	43.04	38.42	3.80	0.16	45.85	65.71	0.57	115.02	39.07	5.00
MAE [38] (IN1K)	<u>68.0</u>	<u>46.1</u>	79.6	41.59	35.83	1.19	<u>0.08</u>	44.18	<u>59.20</u>	0.55	106.08	36.09	<u>2.13</u>
MutliMAE [4] (IN1K)	60.2	46.4	<u>83.0</u>	<u>41.42</u>	35.38	2.17	0.07	<u>44.03</u>	60.35	0.56	105.25	36.17	2.75
MAE (Habitat)	32.5	40.3	79.0	42.06	<u>33.63</u>	1.79	<u>0.08</u>	44.81	59.76	0.56	<u>102.54</u>	<u>35.65</u>	2.88
CroCo (Habitat)	37.0	40.6	85.6	40.91	31.34	<u>1.74</u>	<u>0.08</u>	41.69	54.13	0.55	93.58	33.00	1.25

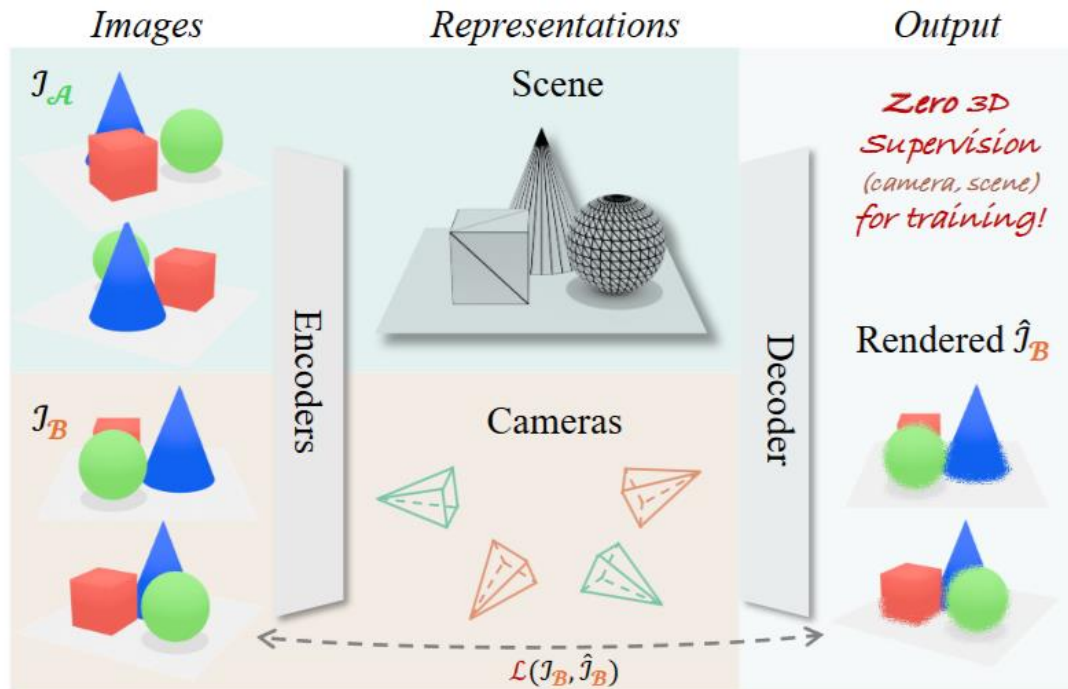
Table 5: **Relative pose estimation results** with the median camera position and orientation errors on 7-scenes. Finetuning a model pre-trained with CroCo achieves competitive results compared to existing methods directly regressing relative camera pose, without applying any fusion technique.

Method / pre-training	chess	fire	heads	office	pumpkin	redkitchen	stairs	Average
RelocNet* [6]	12cm, 4.14°	26cm, 10.44°	14cm, 10.5°	18cm, 5.32°	26cm, 4.17°	23cm, 5.08°	28cm, 7.53°	21cm, 6.74°
NC-EssNet* [96]	12cm, 5.63°	26cm, 9.64°	14cm, 10.66°	20cm, 6.68°	22cm, 5.72°	22cm, 6.31°	31cm, 7.88°	21cm, 7.50°
CamNet* [†] [25]	<u>4cm, 1.73°</u>	3cm, 1.74°	<u>5cm, 1.98°</u>	<u>4cm, 1.62°</u>	4cm, 1.64°	4cm, 1.63°	4cm, 1.51°	4cm, 1.69°
top1 AP-GeM-18	27.9cm, 12.81°	40.4cm, 16.06°	21.6cm, 16.46°	37.5cm, 12.79°	44.4cm, 12.58°	46.7cm, 13.92°	32.2cm, 14.59°	36cm, 14.2°
MAE (Habitat)	13.2cm, 9.44°	32.0cm, 15.10°	16.0cm, 16.75°	24.8cm, 11.54°	25.4cm, 10.62°	29.4cm, 13.32°	32.8cm, 14.88°	24.8cm, 13.09°
CroCo (Habitat)	2.4cm, 2.81°	<u>4.0cm, 3.86°</u>	3.1cm, 4.00°	3.4cm, 2.53°	<u>4.9cm, 2.79°</u>	<u>5.5cm, 3.72°</u>	<u>11.7cm, 4.53°</u>	<u>5.0cm, 3.46°</u>

*: fuse multiple pose predictions

[†]: exploit temporal information and multi-step retrieval

One step further: full 3D from unlabeled data



One step forward: full 3D from unlabeled data

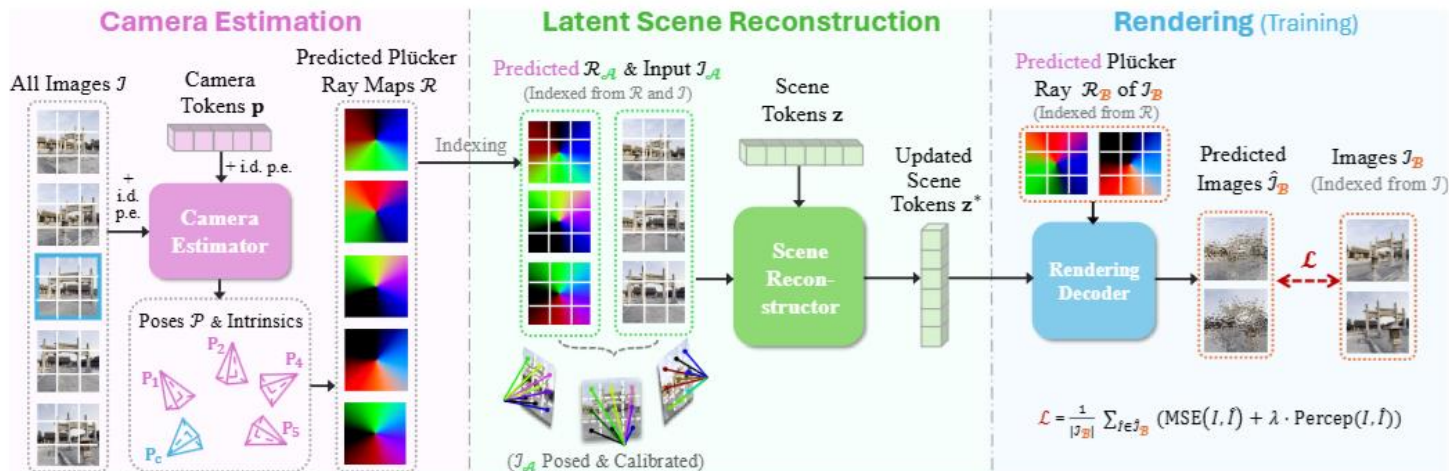


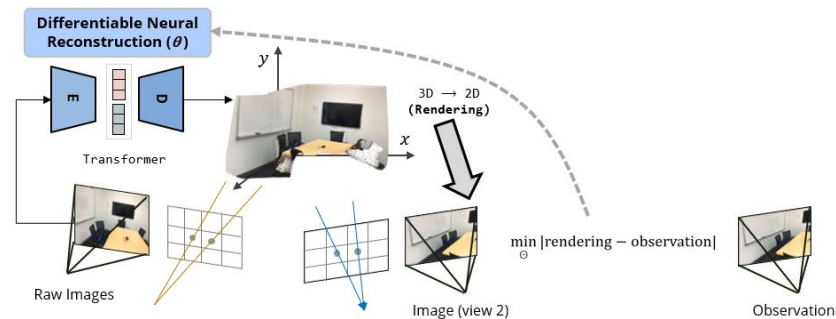
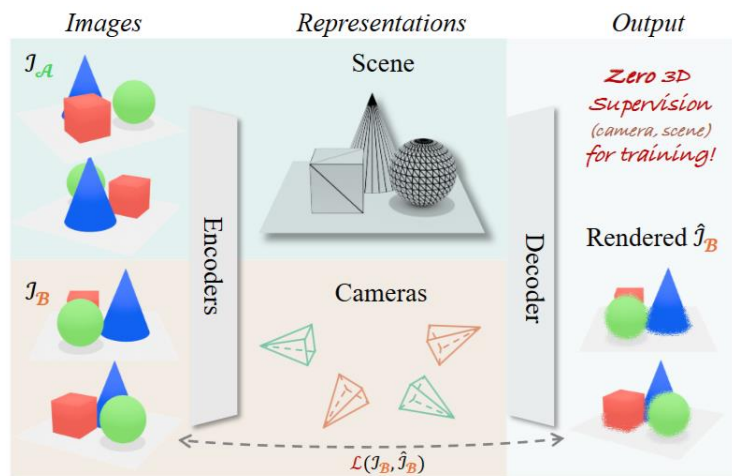
Figure 3. RayZer self-supervised learning framework. RayZer takes in unposed and uncalibrated multi-view images $\mathcal{I}$ and predicts per-view camera parameters and a scene representation, which supports novel view rendering. (Left) RayZer first estimates camera parameters, where one view is selected as the canonical reference view (in blue box). RayZer predicts the intrinsics and the relative camera poses $\mathcal{P}$ of all views. The predicted cameras are then converted into pixel-aligned Plücker ray maps $\mathcal{R}$. (Middle) RayZer uses a subset of input images, $\mathcal{I}_A$, as well as their previously predicted camera Plücker ray maps, $\mathcal{R}_A$, to predict a latent scene representation. Here, the Plücker ray maps, $\mathcal{R}_A$, serve as an effective condition for scene reconstruction. (Right) RayZer can render a target image given the scene representation $\mathbf{z}^*$ and a target camera. During training, we use $\mathcal{R}_B$, which is the previously predicted cameras Plücker ray maps of $\mathcal{I}_B$, to render $\hat{\mathcal{I}}_B$. This allows training RayZer end-to-end with self-supervised photometric losses between inputs $\mathcal{I}_B$ and their renderings $\hat{\mathcal{I}}_B$.

Visualization of the self-supervised 3D representation

We show results on **DL3DV**

(rendering by interpolating predicted SE(3) poses)

Comparisons: Supervised Training vs. Self-Supervised training



Hanwen Jiang, et, al., “RayZer: A Self-supervised Large View Synthesis Model”, ICCV 2025

Zhiwen Fan, et, al., “Large Spatial Model: End-to-end Unposed Images to Semantic 3D”, NeurIPS 2024